CS 300 – Chapter 2 Lectures

Introduction to Algorithms

Jose de Lima

April 23, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 1 taught you how to *type* a loop; chapter 2 teaches you how to *decide which loop to write* when you haven't been given one. This is the bridge between "I can code" and "I can solve a novel problem." Every later chapter is a specific data structure; this chapter is the *meta-skill* of picking an algorithmic strategy in the first place.

What you add to your toolkit here:

- A vocabulary for problems vs. algorithms, and for what makes one algorithm "better" than another (correctness, speed, memory, simplicity).
- A **recursive mindset**: base case + smaller subproblem. Factorial and binary search are the training wheels; BSTs, merge sort, and graph traversal all run on this.
- Three named strategies you'll reach for repeatedly: **heuristic**, **greedy**, **dynamic programming**. Knowing when each is appropriate is half the exam.
- An ethics / data-privacy framework so that the code you write doesn't just *work* but is defensible when someone asks why you chose to collect or expose certain data.

Mastery checklist – you should be able to, without references:

1. State the two ingredients every recursive function needs, and show what goes wrong if either is missing (infinite recursion / wrong answer).
2. Write `factorial(n)` and `binarySearch(v, t)` recursively in C++ *and* iteratively, and argue when each form is appropriate.
3. Explain why naïve recursive Fibonacci is exponential while the iterative / memoized version is linear – in terms of the recursion tree, not hand-waving.
4. Given a new problem, correctly classify it as a candidate for greedy, DP, or neither, and defend the choice with one of the structural tests (greedy-choice property, optimal substructure, overlapping subproblems).
5. Walk through the fractional-knapsack greedy proof ("always pick highest value/weight ratio next") and know why the *0/1* variant is not solvable greedily.
6. Identify overlapping subproblems in an LCS-like problem and write the 2D recurrence relation before touching code.
7. Name at least one privacy or ethics consideration that could change your design decision on a data-driven feature.

Looking ahead. Chapter 3 gives you the formal language (Big-O) to justify the "faster" and "slower" claims you made informally here. The sorting algorithms in ch. 3 are concrete instances of the strategies

here: insertion/selection sort are incremental, merge/quicksort are divide-and-conquer, radix sort is a non-comparison heuristic. Expect ch. 3 to be easier if your ch. 2 recursion intuitions are solid.

1 2.1 Introduction to Algorithms

Chapter 1 covered the *mechanics* of C++ – vectors, strings, loops. Chapter 2 starts asking a different question: given a problem, what sequence of steps actually solves it, and how do you compare one sequence against another? That is the subject of **algorithms**, and it is the real content of this course.

1.1 Problems vs. algorithms

Two words that get used interchangeably but mean different things.

Definition — Computational problem

A **computational problem** is a precise specification of three things: the **input** the solver will receive, the **question** being asked about that input, and the **output** the solver should produce. It describes *what* needs to be done, not *how*.

Definition — Algorithm

An **algorithm** is a finite, unambiguous sequence of steps that solves a computational problem. Algorithms describe *how* the problem gets solved. The same problem can have many algorithms; the interesting question is usually which one is best.

Example A problem and one algorithm that solves it

Problem: Given an array of numbers, what is the maximum value?
Algorithm (pseudocode):

```
FindMax(a):
    max = a[0]
    for i = 1 to a.size - 1:
        if a[i] > max:
            max = a[i]
    return max
```

The problem specification does not mention a loop, or `max`, or `i`. Those are *implementation decisions* the algorithm makes. A different algorithm – e.g., sort the array and return the last element – solves the same problem with different steps and different efficiency.

Aside Algorithms exist independent of language

Algorithms are usually described in pseudocode, diagrams, or English because the language is incidental to the *idea*. The C++ implementation you will write in homework is just one translation of the algorithm. Binary search is binary search whether it is written in C++, Python, or executed by a human with a pencil.

1.2 Why this matters: picking the right algorithm is the job

Most interesting problems you will ever see have already been reduced to a small set of well-known subproblems, each with a well-studied algorithm. Half of being a good engineer is recognizing which known problem the thing in front of you actually is.

Domain	Problem	Common algorithm
DNA analysis	Longest shared subsequence	Longest common substring
E-commerce / search	"Is this product in stock?"	Binary search
Navigation	Fastest route A $\to$ B	Dijkstra's shortest path
Scheduling	Which tasks can run first?	Topological sort
Compilers / build	What to rebuild after a change	DFS on a dependency graph
Spell check / autocomplete	Closest word to a typo	Edit distance (DP)

The CS 300 syllabus is essentially a tour of the right-hand column – sorting, searching, hashing, trees, heaps, graph traversal. Once you can name the tool, the implementation becomes mechanical.

1.3 What makes an algorithm "good"?

The dominant metric is **runtime growth** – how the number of steps changes as the input gets bigger. This is the subject of Chapter 2 proper.

Definition — Efficient algorithm (informal)

An algorithm is **efficient** if its runtime grows at most **polynomially** with the input size: n, $n \log n$, n^2, n^3, etc. Anything exponential (2^n, $n!$) is considered too slow to scale.

Aside Why polynomial is the cutoff

It is an arbitrary line, but a useful one. Polynomial-time algorithms remain tractable as inputs grow – doubling n multiplies the runtime by a constant factor. Exponential algorithms do not – doubling n can *square* the runtime. On a modern machine, 2^{40} is measured in minutes; 2^{60} is measured in years. That is the difference between "run it overnight" and "never finishes."

1.4 NP-complete problems: when no fast algorithm is known

Some problems have resisted every attempt to find an efficient algorithm. These are the **NP-complete** problems.

Definition — NP-complete (intuitive)

A problem is **NP-complete** if:

- nobody has found a polynomial-time algorithm for it,
- nobody has proven that one is impossible, and
- a fast algorithm for *any* NP-complete problem would give a fast algorithm for *all* of them.

Whether **P** = **NP** – i.e., whether such an algorithm exists – is the most famous open problem in computer science.

Example The clique problem

Input: a social-network graph and an integer k.
Question: is there a set of k people who all know each other?

Finding a clique of size 3 in a small graph is easy. Finding one of size 50 in a graph of millions of nodes is, as far as anyone knows, beyond any algorithm that could finish this century. Same problem statement, exponentially harder instance.

Key idea What you do when a problem is NP-complete

Not "give up." The usual response is to solve a *relaxed* version:

- **Approximation:** find a provably-close-to-optimal answer quickly.
- **Heuristic:** use a good-enough rule of thumb that works on typical inputs.
- **Special-case:** if the input has extra structure (sparse graph, small k, ordered input), a faster algorithm may exist for that subset.

Recognizing a problem as NP-complete saves you from hunting for an efficient exact algorithm that doesn't exist. That is a skill in itself.

Key idea Big picture

This course teaches two things simultaneously: a catalog of named algorithms that handle the most common computational problems, and a *language* for comparing algorithms (big-O, which is formalized in chapter 3) so you can justify your choice. When you finish, the goal is not that you have memorized every pseudocode listing, but that you can look at a new problem, say "that's a search on sorted data, so binary search is $O(\log n)$" – and pick up the right tool without rediscovering it.

2 2.2 Problem Solving

Knowing C++ is a prerequisite for writing code, but it is not what makes programs *work*. The hard part – the part people are actually paid for – is **problem solving**: figuring out a methodical sequence of steps that turns the input into the desired output. The programming language is just the notation you write that sequence down in, the way English is the notation a chef writes a recipe down in. Good recipes come from good cooking, not good English.

Definition — Solution approach

A **solution approach** is the idea of how you will solve a problem, stated in words, diagrams, or pseudocode – independent of any programming language. A good solution approach is worked out before any code is written; the code is the final translation, not the design.

2.1 A non-programming example: matching socks

Three kinds of socks sit unsorted in a drawer. You want one matching pair.

Example Two approaches, very different runtimes

Approach A – "try and put back": pull a sock, pull another, compare. If they match, done. Otherwise put the second one back and try again.
Approach B – "pull and keep": pull a sock, then another, then another, stopping only when you have a pair.
Approach A has no guaranteed bound – you could get unlucky for a long time. Approach B is bounded: with k sock types, pulling $k + 1$ socks *must* produce a match (this is the pigeonhole principle). For 3

sock types, four pulls is enough, always.

Aside The pigeonhole principle

If $n+1$ items go into n boxes, some box must contain at least two items. This comes back repeatedly – hash-table collisions, proofs about sorting lower bounds, birthday-paradox arguments – so recognizing it early is worth it. In the sock problem, the "boxes" are the sock types and the "items" are the socks you've pulled.

2.2 A programming-flavored example: sorting name tags

1000 name tags, currently sorted by first name, need to be re-sorted by last name.

Example Two approaches

Approach 1 – "insert each into a new sorted stack": take one tag at a time, walk down a new stack, drop it in the right place. Roughly n^2 comparisons, because each insertion scans everything already placed.
Approach 2 – "bucket then sort each bucket": make 26 sub-stacks, one per starting letter of the last name. Drop each tag into its letter's bucket. Sort each bucket individually. Concatenate the buckets in order.

Key idea This is actually radix sort in disguise

Approach 2 is a hand-executed version of **radix sort** – partition by one "digit" of the key (here, the first letter of the last name), sort within each partition, and recombine. You will meet radix sort formally later in the course. The point here is that the two approaches give the same answer but the work is distributed very differently, and one of them is a named algorithm with well-known properties. Almost every "clever" approach you invent on a whiteboard turns out to be a known algorithm.

2.3 The 64-person greeting problem

An organizer wants every person at a 64-person meeting to greet every other person for 30 seconds. How long does this take?

Example Doing the arithmetic

64 people, each pair meets once, so the number of pairs is $\binom{64}{2} = \frac{64 \cdot 63}{2} = 2016$ pairs. At 30 seconds each, that is $2016 \cdot 30 = 60{,}480$ seconds ≈ 16.8 hours.

Key idea The combinatorial trap

Schemes of the form "everybody does something with everybody else" are $O(n^2)$. They *look* linear in the problem statement ("just have everyone say hi") but scale quadratically in practice. This is the same reason all-pairs-shortest-path takes $O(n^3)$, why bubble sort is $O(n^2)$, and why naively checking every pair of records in a database does not work at a million-row scale. Watch for "every X with every other X" in a problem statement – it's a flag for $O(n^2)$.

2.4 A usable workflow

The approach this course will try to instill, roughly in order:

1. **Read the problem until you can restate it in one sentence** without looking at the prompt. If you cannot, you haven't understood it yet; coding will not help.
2. **Solve a tiny example by hand.** Before writing code, walk through the problem with a three- or four-element input. This usually exposes missing details and suggests an algorithm.
3. **Describe an approach in pseudocode or English.** Steps, not syntax. If the approach doesn't work on paper, C++ will not save it.
4. **Estimate the cost.** Will this scale to the largest input the problem allows? If no, find a better approach before you write any code.
5. **Translate to C++.** Now the language matters. Idiomatic vectors, bounds checking, `.at()` while debugging, algorithms from `<algorithm>` where they fit.
6. **Test with the tiny example from step 2.** If the output does not match, trace the loop (§1.10 methodology).

Gotcha The anti-pattern: code first, think later

The most common beginner mistake is to start typing at step 5, work out the approach through compiler errors, and debug by randomly changing conditions. It *feels* productive (there are always more changes to try) but often results in code that passes the examples you happened to test and fails everything else. The up-front thinking from steps 1–4 takes minutes and saves hours.

Aside Two paths to an algorithm: reduce, or design

The workflow above leaves the question of *which* algorithm implicit. MIT 6.006 frames it as a top-level binary choice:

1. **Reduce to a problem you already know.** If the problem looks like *search*, reach for a search data structure (array, hash table, BST). If *sort*, reach for a sort algorithm (insertion, merge, radix). If *shortest path*, a graph algorithm (BFS, Dijkstra, Bellman-Ford). The cs-300 syllabus is essentially this catalog of named problems and their canonical solutions.
2. **Design your own (recursive) algorithm.** If no known problem fits, you're inventing. The strategies in §2.4–§2.10 (recursion, divide-and-conquer, DP, greedy, heuristics) are the design patterns for that case.

The reduce path is faster when it applies, which is most of the time on a coursework or job-interview problem. Train the recognition reflex first; the design strategies are what you fall back on when recognition fails.

Key idea Big picture

A programming language is a notation for expressing solutions; it is not itself a solution. The thing you develop in this course is the habit of looking at a problem, recognizing a structure you have seen before, picking the matching algorithm, and only then writing the C++. When you get stuck on a homework problem, the fix is almost never "more C++" – it is going back to pseudocode or walking

through a 3-element input by hand.

3 2.3 Computational Problem Solving

§2.2 framed problem solving in general terms. This section narrows in on the specific style of thinking that computing demands, which cognitive-science and CS-education literature has settled on calling **computational thinking**. It is four habits – decomposition, pattern recognition, algorithms, and abstraction – applied together. The names are generic; the habits are not.

3.1 The input > process > output model

Every program, no matter how large, fits the same shape at the top level: take input, compute, produce output (and/or persistent side effects). Keeping this shape in mind while designing prevents the common trap of conflating "how data gets in" with "what we do with it."

Example Primality check, top-level view

```
INPUT:    a positive integer n
PROCESS:  decide whether n has any divisors other than 1 and n
OUTPUT:   "prime" or "not prime"
```

The process step is where the algorithm lives. The input and output steps are routine – read an int, print a string – and should not dominate your design time.

3.2 The four habits of computational thinking

Definition — Computational thinking

A problem-solving approach built from four interacting moves:

- **Decomposition** – break a big problem into smaller pieces.
- **Pattern recognition** – notice repeated structure within or across problems.
- **Algorithms** – express the solution as a precise sequence of steps.
- **Abstraction** – generalize so one solution handles a whole family of problems.

These are not stages in a waterfall. They feed each other: decomposing reveals patterns, patterns suggest algorithms, a finished algorithm gets abstracted so it can be reused.

3.3 Worked example: is n prime?

Watch the four habits in action on one problem.

Decomposition.. Big question ("is n prime?") into smaller yes/no questions: is it divisible by 2? by 3? by 4? … If any answer is yes, n is not prime; if all are no, it is prime.

Pattern recognition.. Two useful observations:

- We can stop the first time we see a divisor – we don't need to check the rest.
- No factor of n (other than n itself) can exceed $n/2$, so we never need to test past there. In fact, no factor exceeds $\sqrt{n}$, which is a much tighter bound.

Algorithm..

```
IsPrime(n):
    if n < 2: return false
    for d = 2 to sqrt(n):
        if n mod d == 0: return false
    return true
```

Abstraction.. The algorithm doesn't depend on any specific value of n; it works for every positive integer. Expressed as a function it becomes a reusable tool.

Example C++ translation

```
#include <cmath>

bool is_prime(int n) {
    if (n < 2) return false;
    int limit = static_cast<int>(std::sqrt(n));
    for (int d = 2; d <= limit; ++d) {
        if (n % d == 0) return false;
    }
    return true;
}
```

Three pattern-recognition wins compound here: stop at `sqrt(n)` instead of $n/2$, return as soon as a divisor is found, and handle the $n < 2$ edge case up front so the loop body doesn't have to.

Aside Why $\sqrt{n}$ is enough

If $n = a \cdot b$ with $a \leq b$, then $a \leq \sqrt{n}$. So if n has any factor at all, it has one no bigger than $\sqrt{n}$. Checking past $\sqrt{n}$ is redundant. Turning a $O(n)$ primality check into $O(\sqrt{n})$ is not a rewrite; it's a single-line loop bound change that pattern-recognition hands you for free.

3.4 Second example: sum of 1 to 100

Decomposition.. Add numbers one at a time: $1 + 2$, then $+3, \ldots$

Pattern recognition.. Pair the numbers from the ends in: $(1 + 100), (2 + 99), (3 + 98), \ldots$ Each pair sums to 101, and there are 50 pairs. Total: $50 \cdot 101 = 5050$.

Algorithm.. The closed-form $\dfrac{n(n+1)}{2}$ computes the sum in $O(1)$ rather than $O(n)$.

Abstraction.. The formula works for any positive n, not just 100. It generalizes further to arithmetic sequences.

Example Two implementations, same output

```
// O(n) loop -- decomposition only
int sum_loop(int n) {
    int s = 0;
```

```
    for (int i = 1; i <= n; ++i) s += i;
    return s;
}

// O(1) closed form -- pattern recognition plus algebra
int sum_formula(int n) {
    return n * (n + 1) / 2;
}
```

For $n = 100$ both return 5050. For $n = 1{,}000{,}000{,}000$, only the second one finishes in microseconds.

Key idea Pattern recognition is the most valuable habit

Of the four, pattern recognition is where the real leverage lives. Decomposition and algorithms are mechanical once the pattern is clear; abstraction is mostly cleanup. Spotting that "this is a sum of pairs" or "this is a divisibility scan" turns a tedious loop into a one-line expression, and recognizing "this is binary search" turns an $O(n)$ scan into $O(\log n)$. Invest in the vocabulary of named patterns – this course is partly that vocabulary.

3.5 The loop from §2.2 revisited

The workflow from the previous section is computational thinking in practice:

Workflow step	CT concept
Restate the problem	(prerequisite)
Solve a tiny example by hand	decomposition
Spot the structure	pattern recognition
Write pseudocode	algorithm
Generalize to a function	abstraction
Estimate the cost	pattern recognition (again)
Translate to C++	implementation
Test on the tiny example	verification

Key idea Big picture

Computational thinking is not a separate subject from programming; it is the shape of the thinking programming rewards. Every named algorithm in this course – binary search, merge sort, Dijkstra, hash tables – is someone's decomposition-plus-pattern-recognition written down and abstracted. The more of them you know, the less original design you need to do on any given problem: you recognize the shape, retrieve the matching algorithm, and translate it. That is what makes experienced engineers productive.

4 2.4 Recursive Definitions

A **recursive** algorithm solves a problem by reducing it to a *smaller instance of the same problem* and combining the answer. It is the same mental move as mathematical induction, and it unlocks a surprising number of problems that are awkward to express with loops alone – tree traversals, divide-and-conquer sorts, graph searches, backtracking puzzles. The entire back half of this course leans on it.

Definition — Recursive algorithm

An algorithm is **recursive** if it solves a problem by (1) identifying one or more **base cases** that can be answered directly, and (2) reducing every other input to a *smaller* instance of the same problem, calling itself on the smaller instance, and combining the result.

Definition — Base case

The **base case** is the one (or few) input(s) that the algorithm answers without recursing. Every recursive algorithm needs at least one. Without it, the algorithm keeps reducing forever – in code, the call stack grows until the program crashes.

Definition — Recursive case

The **recursive case** is the work the algorithm does on a non-base input: reduce the problem, call itself on the smaller version, combine the returned answer with whatever local work remains.

4.1 The anatomy of every recursive function

Example The template

```
ReturnType solve(Input x) {
    if (is_base_case(x)) {
        return direct_answer(x);        // base case
    }
    Input smaller = reduce(x);          // make progress
    ReturnType sub = solve(smaller);    // recurse
    return combine(x, sub);             // combine and return
}
```

Every recursive function you will write is some fill-in-the-blank version of this skeleton.

Definition — SRTBOT: a fuller version of the template

The 3-part template above (*base, reduce, combine*) is the core of recursive algorithm design. MIT 6.006 expands it into a 6-step recipe – mnemonic **SRTBOT** – that adds the explicit checks the abbreviated template leaves implicit:

1. **S**ubproblem definition. State the meaning of a subproblem in words, in terms of parameters. Often subsets of input: prefixes, suffixes, contiguous substrings.
2. **R**elate. Express the subproblem solution in terms of smaller subproblems. The design pattern: *identify a question whose answer would reduce to smaller subproblems, then locally brute-force all possible answers.*
3. **T**opological order. Argue the relation is acyclic – subproblems form a DAG – so the recursion terminates.
4. **B**ase cases. Solutions for the smallest subproblems where the relation breaks down.
5. **O**riginal problem. Show how to compute the answer to the original problem from subproblem solutions. Use parent pointers to recover the actual solution, not just the objective value.
6. **T**ime analysis. Sum work over subproblems ($\sum_{x \in X} \text{work}(x)$, treating recursive calls as $O(1)$).

The 3-part template is the SRTBOT core: *base* = step 4, *reduce + combine* = step 2. SRTBOT pays

off when you start writing dynamic-programming algorithms in §2.10, where the topological-order argument and the work-per-subproblem analysis become the actual content of the design.

4.2 Example 1: factorial

$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1$, with $1! = 1$ as the base.

Example Factorial in C++

```
long long factorial(int n) {
    if (n <= 1) return 1;             // base case
    return n * factorial(n - 1);      // recursive case
}
```

Trace for $n = 4$:

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * 2 * factorial(1)
             = 4 * 3 * 2 * 1
             = 24
```

Gotcha `n!` overflows 64-bit `long long` at $n = 21$

$20! \approx 2.4 \times 10^{18}$, just under the `long long` max. $21!$ silently wraps around to a nonsense value. For bigger inputs you need a big-integer type or modular arithmetic. This isn't a recursion bug – iterative factorial has the same problem – but students often think the recursive version "broke" when the arithmetic did.

4.3 Example 2: cumulative sum

Sum of $0 + 1 + 2 + \cdots + n$. Base case: sum to 0 is 0.

Example `sum(n)`

```
int cumulative_sum(int n) {
    if (n == 0) return 0;
    return n + cumulative_sum(n - 1);
}
```

$\texttt{sum}(4) = 4 + 3 + 2 + 1 + 0 = 10$.

This is a great "first recursion" exercise, but the closed form $n(n+1)/2$ is O(1) and doesn't risk stack overflow. Recursion here is pedagogy, not the right tool.

4.4 Example 3: reverse a vector by swapping from both ends

A recursive rewrite of the reversal loop from §1.9.

Example Reverse in place, recursively

```
void reverse_list(std::vector<int>& v, int lo, int hi) {
    if (lo >= hi) return;              // base: 0 or 1 element left
    std::swap(v[lo], v[hi]);           // work at this layer
```

```
    reverse_list(v, lo + 1, hi - 1);   // recurse on the inside
}

// Caller:
reverse_list(v, 0, v.size() - 1);
```

Compare to the loop in §1.9. The recursion mirrors the structural invariant directly: *the outer layer is done; reverse the inner layer.* Some programmers find that clearer than the `v.size() / 2` loop bound.

4.5 What can go wrong

Gotcha 1. Missing or unreachable base case

```
int factorial(int n) {
    return n * factorial(n - 1);   // never stops
}
```

`factorial(0)` calls `factorial(-1)` calls `factorial(-2)` … the program crashes with a stack overflow. Every recursive function needs a base case that's *reachable* from every legal input.

Gotcha 2. Recursing on the same or larger subproblem

```
int f(int n) {
    if (n == 0) return 0;
    return f(n) + 1;              // same n -- infinite recursion
}
```

If the recursive call isn't on a *strictly smaller* input (or strictly closer to the base case), recursion doesn't terminate. A useful habit: write down a *measure* – an integer that decreases on every call – and confirm it reaches the base.

Gotcha 3. Stack overflow on deep recursion

C++ call stacks are usually measured in megabytes; each stack frame is a few dozen to a few hundred bytes. That makes the practical recursion depth somewhere in the tens of thousands to millions of frames before the stack blows. For chains like `factorial(n)` or recursive sum this rarely matters, but for tree traversal on a badly-shaped tree – or for code that recurses on linked list length – you can hit it. Iteration or explicit stack data structures are the fixes.

Aside Recursive strategies, classified by call-graph shape

A recursive call graph (vertex per call, directed edge $A \to B$ if B calls A) must be acyclic for the recursion to terminate. The *shape* of that DAG corresponds 1:1 with named algorithm-design strategies you'll meet in §2.5–§2.10:

Strategy	Call-graph shape
Brute force	Star (all calls at one level)
Decrease & conquer	Chain (each call has one child)
Divide & conquer	Tree (calls split, no overlap)
Dynamic programming	DAG (calls overlap; reuse via memo)
Greedy / incremental	Subgraph (only one path explored)

This is the unifying framing for the rest of the chapter. §2.5 binary search is decrease-and-conquer (chain); naive Fibonacci is divide-and-conquer with overlap (a tree that *wants* to be a DAG); §2.10

DP is the corresponding DAG fix; §2.9 greedy commits to one branch instead of exploring the whole shape. When you can sketch the call graph for a small input, you can usually name the strategy by inspection.

4.6 When to reach for recursion

Key idea Recursion shines on recursive structure

Some data has recursion baked in – **trees** have subtrees, **lists** have tails, **grids** have sub-grids, **graph searches** explore neighbors whose structure is "the same problem on a smaller set of nodes." When the data is recursive, recursive code is shorter and clearer than the iterative equivalent. When the data is linear and the work is constant-per-element, loops almost always win.

4.7 Recursion and induction – the same idea

A correctness argument for recursion mirrors a proof by induction:

1. **Base case.** The base-case answer is correct by direct inspection.
2. **Inductive step.** Assume the recursive call returns the correct answer for the smaller input. Show that the combine step produces the correct answer for the current input.

Example Correctness of `factorial`

Base: `factorial(1) == 1`, matching $1! = 1$.
Step: Assume `factorial(n-1) == (n-1)!`. Then `n * factorial(n-1) == n * (n-1)! == n!`. By induction, `factorial(n)` is correct for all $n \geq 1$.

Key idea Big picture

Recursion is worth the investment even when a loop would work, because half of the algorithms in the rest of this course are most naturally expressed recursively: binary search, merge sort, quicksort, tree traversal, DFS, backtracking, dynamic programming recurrences. Get comfortable with the three-part template (base, reduce, combine) and the induction-style correctness argument, and those algorithms stop feeling mysterious – they are the same pattern applied to different kinds of "smaller subproblem."

5 2.5 Recursive Algorithms

§2.4 introduced the mechanics of recursion on small arithmetic examples. This section applies the same three-part template (base, reduce, combine) to two algorithms you will actually use: **Fibonacci** (a warning sign about naive recursion) and **binary search** (a textbook win for recursion). Read them side by side – same shape, wildly different efficiency.

5.1 Fibonacci: the cautionary tale

Definition — Fibonacci sequence

$F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. The sequence begins $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots$

The definition *is* a recurrence, so the recursive implementation writes itself:

Example Naive recursive Fibonacci

```
long long fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}
```

Base cases: $n = 0$ and $n = 1$. Recursive case: sum of two smaller subproblems.

Gotcha This function is catastrophically slow

`fib(50)` takes minutes. `fib(60)` won't finish today. The reason: `fib(n)` calls both `fib(n-1)` and `fib(n-2)`, which each call two more, and so on. The number of calls grows roughly like $\phi^n \approx 1.618^n$. The recursion tree has billions of nodes, almost all of them computing values you've already computed:

```
                fib(5)
               /      \
          fib(4)       fib(3)
         /     \      /     \
      fib(3)  fib(2) fib(2) fib(1)
      ...
```

`fib(3)` is computed twice, `fib(2)` three times, `fib(1)` five times. This is the canonical example of a *correct* recursive algorithm that is *unacceptably slow*.

Key idea Two standard fixes

Memoization: remember each result the first time you compute it. Recursion plus a cache turns $O(\phi^n)$ into $O(n)$.

Iteration (bottom-up): just build the sequence from the bottom in a loop – no recursion, $O(n)$ time, $O(1)$ space.

```
long long fib_iter(int n) {
    long long a = 0, b = 1;
    for (int i = 0; i < n; ++i) {
        long long next = a + b;
        a = b;
        b = next;
    }
    return a;
}
```

The *shape* of the recurrence makes naive recursion tempting. Recognizing overlapping subproblems – a job for pattern recognition (§2.3) – tells you to cache or iterate instead.

Aside This is a preview of dynamic programming

"Recursion with overlapping subproblems" is the single-sentence definition of **dynamic programming**. Every DP problem is a recursive definition plus a cache (memoization) or a loop (tabulation). You will meet this idea formally later; the Fibonacci blow-up is where the need for it first becomes obvious.

5.2 Binary search: the textbook win

Here recursion earns its keep. Binary search on a *sorted* array finds a key in $O(\log n)$ comparisons – dramatically faster than the linear $O(n)$ scan from §1.5.

Definition — Binary search

To search a sorted array for a key: look at the middle element. If it equals the key, return. If the key is smaller, recurse on the left half; if larger, recurse on the right half. If the range becomes empty, the key is not present.

Example Recursive binary search

```
int binary_search(const std::vector<int>& a, int lo, int hi, int key) {
    if (lo > hi) return -1;                          // base: empty range
    int mid = lo + (hi - lo) / 2;                    // avoid overflow
    if (a[mid] == key)  return mid;
    if (a[mid] < key)   return binary_search(a, mid + 1, hi, key);
    else                return binary_search(a, lo, mid - 1, key);
}

// Caller:
int idx = binary_search(v, 0, v.size() - 1, 42);
```

Gotcha `mid = (lo + hi) / 2` is an overflow bug

If `lo + hi` exceeds `INT_MAX`, the sum wraps negative and `mid` lands somewhere random. The fix is `mid = lo + (hi - lo) / 2`, which computes the same value without ever adding two large indices. This bug sat undiscovered in the JDK's `Arrays.binarySearch` for nearly a decade (Josh Bloch, 2006). Make the safe form muscle memory.

5.3 Tracing binary search on a concrete input

Example Search for 42 in (14, 26, 42, 59, 71, 88, 92)

```
bs(v, 0, 6, 42):   mid = 3,  v[3] = 59.  59 > 42  ->  search left
bs(v, 0, 2, 42):   mid = 1,  v[1] = 26.  26 < 42  ->  search right
bs(v, 2, 2, 42):   mid = 2,  v[2] = 42.  found. return 2
```

Three comparisons to find the key in a 7-element list. A linear scan would also take 3 here, but scale the list to 10^9 elements: linear is $\sim 10^9$ comparisons, binary is ~ 30. That's the exponential-vs-logarithmic gap.

Aside Why $O(\log n)$

Each recursive call halves the remaining range. Starting from n elements, you reach a range of size 1 after $\log_2 n$ halvings. 7 elements $\to \log_2 7 \approx 3$ steps. 10^9 elements $\to 30$ steps. Binary search is the archetypal divide-and-conquer algorithm.

5.4 Preconditions and edge cases

Gotcha The array MUST be sorted

Binary search on an unsorted array silently returns wrong answers, not an error. If the data isn't already sorted, either sort first (which costs $O(n \log n)$ and only pays off when you will search many times) or use linear search.

Gotcha The off-by-one on the recursive range

The recursive calls use `mid + 1` and `mid - 1`, not `mid`. Forgetting the ± 1 leaves `mid` in the remaining range forever, and when `a[mid]` keeps failing to match, the recursion never terminates. This is exactly the "make the subproblem strictly smaller" rule from §2.4.

Aside `std::binary_search` and `std::lower_bound`

The standard library already ships binary search, and the iterative version it uses is somewhat faster in practice:

```
#include <algorithm>
bool found = std::binary_search(v.begin(), v.end(), key);

auto it = std::lower_bound(v.begin(), v.end(), key);
// it points to the first element >= key
```

Use these in production code; write the recursion by hand to understand it.

5.5 Side-by-side: when recursion helps and when it hurts

	Fibonacci (naive)	Binary search
Recursive calls per step	2	1
Subproblem overlap	massive	none
Time complexity	$O(\phi^n)$	$O(\log n)$
Typical depth	n	$\log n$
Stack risk at $n = 10^6$	overflow long before	safe (depth ~ 20)

Key idea Big picture

Recursive definitions are free; efficient recursive algorithms are not. When the recursion tree has *branching plus overlap*, you pay exponentially – Fibonacci is the small-but-brutal example. When each call strictly shrinks the problem and doesn't duplicate work, recursion is often the clearest possible code – binary search is the archetype. A useful habit when writing a recursive function: sketch the call tree for a small input. If the tree is a single chain or a balanced split, the algorithm is fine. If you see the same subproblem appearing on multiple branches, stop and think about memoization or a loop before writing code.

6 2.6 Data Privacy

This section interrupts the algorithms thread to cover professional ethics – specifically, how a working developer should think about confidential data. The material is short on C++ but the habits it teaches show up in every real job: which data you collect, where you store it, who can see it, and how to reason about the tradeoffs when those choices conflict.

Definition — Confidential data

Confidential data is any data whose exposure could harm the people or organization it describes – names, home addresses, Social Security numbers, health records, passwords, financial records, written communications, location history. The baseline assumption must be that unauthorized people will at some point try (or accidentally manage) to see it.

Definition — Data breach

A **data breach** is any incident in which confidential data is exposed to an unauthorized party – by external attack, insider misuse, accidental disclosure, physical theft, or lost hardware. Breaches are rarely caused by a single dramatic failure; they're the compounding effect of many small lapses.

6.1 "Alertness" as a professional habit

The textbook framing borrows a line from safety engineering: the worst failures are the ones a little alertness would have prevented. For a developer, "alertness" translates to a handful of reflexes:

- Before writing code that collects a piece of data, ask whether it's actually needed.
- Before sending a file, email, or dataset, ask who else sees it along the way.
- Before stepping away from a laptop with sensitive data open, lock the screen.
- Before committing, check for credentials, keys, or customer data in the diff.

None of these are technical. All of them are prevented by a quick pause, and none of them take a second when they become habit.

6.2 The Company XYZ scenario (and how it should have gone)

The textbook scenario: a data analyst is hired to do a confidential job-satisfaction survey. Data collected: name, home address, supervisor, free-form criticism. Data stored: an unencrypted Excel spreadsheet on a laptop left unlocked in a break room. Data leaked: the spreadsheet, forwarded by an admin who saw it open. Fallout: harassment, lawsuit, resignations.

Example Four separate failures, any one of which would have prevented it

- **Collected too much.** Home address wasn't needed for "satisfaction correlated with commute" – ZIP code or commute-time bucket would have worked.
- **Stored it identified.** Responses could have been anonymized at intake (swap name for a random ID, discard the mapping after collection closes).
- **Stored it in the clear.** The file should have been encrypted at rest and protected by a strong password.
- **Left the device unlocked.** A password-locked screen and a full-disk-encrypted laptop would have kept the casual snooper out.

Each of these is minutes of work. Together they turn a career-ending breach into a non-event.

6.3 The two levers: collection and release

Definition — Minimize collection risk

Collect the *least* data that answers the question. If a summary suffices, collect the summary. If an anonymous identifier suffices, don't collect the name. Data that was never collected cannot leak.

Definition — Minimize release risk

Treat every piece of collected data as if it will eventually be exposed. Encrypt at rest, lock access behind authentication, partition storage so a single breach doesn't expose everything, and delete data when it's no longer needed.

Aside The eBay and Target breaches show the second principle at work – partially

eBay's 2014 breach exposed names, addresses, birth dates, and encrypted passwords *because financial data was stored separately* and therefore did not leak with the rest. Target's 2013 breach did include credit-card numbers because the same segmentation was not in place. Same kind of attack; different architectural decisions about what data lives where; very different outcomes.

6.4 The tradeoffs are real

There is no zero-risk data practice. Every privacy protection costs something – inconvenience, reduced analytic power, slower development velocity – and the job is to *consciously* balance cost against risk.

Precaution	Tradeoff
Drop identifying fields	Lose some analyses (commute distance, per-supervisor complaint patterns).
Encrypt the file	Extra step every time you open it; password must be remembered and rotated.
Collect fewer responses	Smaller sample, less statistical power.
Refuse to share a dataset	Slows collaboration; may block a legitimate use.
Retain data longer	More value for longitudinal analysis, more exposure if breached.

Key idea A four-question checklist

Before collecting, storing, or sharing anything sensitive:

1. **Is this analysis worth the exposure risk if this data leaks?**
2. **Can I avoid collecting (or can I anonymize) the sensitive fields?**
3. **Is this data protected at rest** – encryption, access control, locked device?
4. **If someone else wants this data, do they *need* it – or would a summary suffice?**

These four questions catch most breaches before they happen.

6.5 A short history of things that went wrong

The scale of real breaches is hard to picture:

2013, Target	70M+ customer records including credit-card data; stemmed from a compromised HVAC vendor login.
2013, Snowden / NSA	Thousands of classified documents leaked by an insider with legitimate access.
2014, eBay	148M accounts; passwords encrypted but email and address in the clear.
2014, Sony Pictures	Terabytes including emails, salaries, and HR data; a senior executive lost their job over quoted emails.
2015, Anthem	80M records including SSNs and health info.
2015, Ashley Madison	30M user identities of a service whose entire value was discretion; several suicides attributed in part to the exposure.
2017, Equifax	147M Americans' SSNs and credit data; root cause a known Apache Struts vulnerability left unpatched.
2021, Facebook	533M user records including phone numbers scraped via a contact-import misuse.

Aside The pattern in every row

Large, well-resourced companies with security teams still got breached. The common thread isn't "they were incompetent" – it's that attack surface is huge, humans make small mistakes, and modern datasets are valuable enough that attackers are patient. Assuming breach is the default working assumption; the question is how much damage a breach would do, not whether one will ever happen.

6.6 What this means for you on CS 300 and beyond

Course projects rarely handle real PII, but the habits are worth forming now:

- Don't commit credentials, API keys, or `.env` files to git. Add them to `.gitignore` before you run your first `git add`.
- Use environment variables or a secrets manager for API tokens – never string literals in source.
- When a program reads user data, think about where it lives after the program exits: in memory? in a temp file? in a log?
- Treat student test data with the same care you would real data, so the habits hold when the data does matter.

Key idea Big picture

Every algorithm in this course is a *means*, and data is what the algorithms operate on. Efficiency and correctness are necessary but not sufficient – correctness on the wrong data, efficiency at the wrong scope, or a correct analysis leaked to the wrong audience can all cause real harm. Professional responsibility isn't a separate subject from engineering; it's the part of engineering that keeps your correct code from becoming a liability.

7 2.7 Ethical Guidelines

§2.6 covered the narrow question of keeping confidential data confidential. This section widens the frame: once you have data and are doing something with it, what obligations come with the work? The textbook reaches for the American Statistical Association's **Ethical Guidelines for Statistical Practice**, which is a good reference point even though CS 300 is not a statistics course – every engineer who ships a system that affects people is doing applied statistics whether they call it that or not.

Definition — Ethics (working definition)

Ethics are the principles that guide decisions about what is good for individuals and society. Professional ethics are the specific rules a field commits to so that its members can be trusted to do their work responsibly – analogous to the Hippocratic oath in medicine or a building code in civil engineering.

Definition — Data analytics

The practice of examining data to draw conclusions, build models, or inform decisions. Results are often used by people who did not perform the analysis and cannot independently verify it, which is why transparency and integrity matter so much.

7.1 The one-line summary

The ASA opens its guidelines with a sentence worth memorizing:

> Good statistical practice is fundamentally based on **transparent assumptions**, **reproducible results**, and **valid interpretations**.

Those three phrases are the compressed form of everything that follows. When a result looks impressive, ask: were the assumptions stated? can someone else reproduce it? does the interpretation actually follow from the numbers?

7.2 The five categories

The ASA guidelines organize obligations into five buckets. Roughly: what you owe your work, your data, your audience, your subjects, and your field.

1. Professional integrity and accountability..

- Pick the sampling and analytic approach that actually answers the question, not the one that gives the answer you wanted.
- Identify and reduce sources of bias. Disclose what you can't eliminate.
- Disclose conflicts of interest – financial, political, or otherwise. Readers get to discount your work accordingly.

2. Integrity of data and methods..

- Document what you did. Not "document in principle" – the preprocessing, the filters, the parameters, the cutoffs.
- Report results truthfully, including the uncertainty. "Our model is 94% accurate" is meaningless without a confidence interval, a baseline, and the data it was measured on.
- Share data and code when you can, redacting or aggregating to preserve privacy when you can't.

3. Responsibilities to the public and/or client..

- Explain the tradeoffs of the approach you chose, including what it cannot tell them.
- Make useful knowledge broadly available when it doesn't compromise privacy or legitimate confidentiality.

4. Responsibilities to research subjects..

- Stay current on best practices for protecting the people whose data you hold.
- Protect privacy and confidentiality of subjects even when doing so costs you analytic power.

5. Allegations of misconduct..

- Don't condone – or cover for – bad practice by colleagues.
- Distinguish honest error and genuine disagreement from misconduct. They look similar and are judged very differently.
- Don't retaliate against whistleblowers.

7.3 Connecting the categories to CS 300 work

"Statistical practice" sounds distant from a sophomore algorithms class. It shouldn't. A few concrete mappings:

Guideline	What it looks like in CS 300 / future work
Transparent assumptions	Document your time complexity, your input size assumptions, and what "works" means (average? worst case? specific distribution?).
Reproducible results	Fixed random seeds in tests, deterministic hash-table iteration, version-pinned dependencies.
Valid interpretation	"Sort X finished 2× faster" is a claim about the specific input, machine, and compiler flags – not a universal truth.
Disclose conflicts	If you generated your test data with the same code you're benchmarking, say so.
Protect subjects	Don't commit student records, user IDs, or real customer data to your course repo.
Don't retaliate, don't condone	If a teammate ships a function that rounds metric values "to look better," call it out in review.

7.4 The classic ways analyses go bad

Unethical analysis often doesn't look like fraud – it looks like a slightly wrong choice at each of five steps that compound into a misleading conclusion.

Gotcha *p*-hacking / selective reporting

Running many analyses and reporting only the ones that came out "interesting." The reported result looks significant; the underlying process was essentially random. Pre-register the analysis you intend to run before you look at the data, and report all runs, not just the one that worked.

Gotcha Cherry-picked baselines and time windows

"Model X improved accuracy 20%" – compared to what, and over which time window? Picking the weakest baseline or the favorable date range can make an unchanged algorithm look transformative. The mitigation is always to report the alternatives you considered and why you chose this one.

Gotcha Disclosing conclusions, hiding methods

A result that cannot be reproduced is not a result. If a paper, blog post, or dashboard gives you a number without enough detail to recompute it, treat the number as a claim, not evidence.

Gotcha Proxy variables for protected attributes

A model that doesn't take race/gender/etc. as an input but uses ZIP code, first name, or browsing history can reproduce discrimination with a clean deniability story. The ethical question is not what features you included but what outcomes the system produces across groups.

7.5 A short CS 300 reproducibility checklist

Even in course projects, form the habits now:

- **Seed** any randomness you use in benchmarks or tests.
- **Record** compiler, flags, CPU, and input size alongside every performance number.
- **Version** the dataset. "The bids.csv we were given" is not enough – keep a hash or a dated copy.
- **Rerun** the whole pipeline from scratch occasionally to make sure it still produces the reported results.
- **Commit** the analysis code next to the numbers it produced, not separately.

Key idea Big picture

Ethical practice is not a separate module bolted onto "the real work." The three pillars – transparent assumptions, reproducible results, valid interpretations – are the same hygiene that makes engineering teams productive: code you can rerun, tests you can trust, and claims that survive scrutiny. A benchmark nobody else can reproduce is closer to marketing than to engineering; a model whose behavior across subgroups nobody bothered to check is a bug waiting to surface. Treat ethics as a quality bar you hold your own work to, and the ASA guidelines stop feeling like an outside constraint and start feeling like the definition of "done."

8 2.8 Heuristics

§2.1 ended on a note: some problems (the NP-complete ones) have no known efficient exact algorithm. What do you do when you still have to ship? You use a **heuristic** – an algorithm that deliberately trades optimality for speed or simplicity. Heuristics are everywhere in production code: route planners, ranking systems, allocators, schedulers, compilers, game AI. When the exact answer is too expensive to compute, a provably-pretty-good answer in a reasonable amount of time is the whole game.

Definition — Heuristic

A **heuristic** is a technique that willingly accepts a non-optimal or approximate solution in order to improve execution speed, simplicity, or memory use. A heuristic is not a "wrong" algorithm – it's a conscious tradeoff. The tradeoff is the point.

Definition — Heuristic algorithm

An algorithm whose design is driven by a heuristic. It typically runs much faster than an optimal algorithm on the same problem, and typically returns a solution that is *close to* optimal most of the

time – with no guarantee, or a weaker guarantee, of hitting the best possible answer.

8.1 The knapsack problem: the canonical example

Definition — 0-1 knapsack problem

Given a knapsack with maximum weight capacity W and a list of items, each with a weight w_i and a value v_i, choose a subset of items (each included at most once – hence **0-1**) whose total weight is $\leq W$ and whose total value is as large as possible.

Aside Why this is hard

With n items, there are 2^n possible subsets. Checking all of them is exact but exponential. 30 items is already $2^{30} \approx 10^9$ combinations; 50 items is 10^{15}. This is the textbook example of "correct algorithm exists, but good luck running it." 0-1 knapsack is NP-hard; no polynomial-time exact algorithm is known.

8.2 A greedy heuristic for knapsack

Example Greedy by value

```
Knapsack01_greedy(items, capacity):
    Sort items in descending order of value
    remaining = capacity
    chosen = empty list
    for each item in sorted order:
        if item.weight <= remaining:
            add item to chosen
            remaining -= item.weight
    return chosen
```

Take the highest-value item that still fits; keep going until nothing else fits. One pass, plus the sort: $O(n \log n)$ total.

Gotcha This is fast, and sometimes wrong

Capacity 20, items:

Item	Weight	Value
A	15	\$95
B	12	\$60
C	8	\$42

Greedy picks A (\$95, remaining capacity 5) and then nothing else fits. Total: \$95. The optimal answer picks B + C: weight 20, value \$102. The heuristic missed the best answer by choosing the tempting-looking big-ticket item early.

Key idea Smarter greedy: value per pound

Sorting by *value/weight ratio* instead of raw value usually does better, because it packs the knapsack more densely. It still isn't optimal on every instance – the same pathological example can be constructed

against any simple greedy rule – but on random instances it's typically within a few percent of optimal. Choosing the right heuristic measure is half the design work.

Aside The other two options: DP and branch-and-bound

If all weights are integers and W is not too large, 0-1 knapsack can be solved *exactly* in $O(n \cdot W)$ time with dynamic programming. For large W or continuous weights, branch-and-bound prunes the exponential search tree but retains the correct answer. The full story of "when to use heuristic vs. DP vs. branch-and-bound" is beyond this section, but knowing that all three exist helps you reach for the right tool.

Aside "DP solves knapsack" is true – but only *pseudopolynomially*

The line above says 0-1 knapsack can be solved in $O(n \cdot W)$ time with DP, which sounds polynomial. It's not – not in the sense algorithm theorists mean.

Strongly polynomial time is bounded by a constant-degree polynomial in input size measured *in words*. Knapsack's input is n items plus the integer W – so input size is $O(n)$ words. But W itself is one of those words: it can be as large as 2^w where w is the word size. So $O(n \cdot W)$ can be $O(n \cdot 2^w)$ – exponential in input *bits*.

This is called **pseudopolynomial**: bounded by a polynomial in input size *and* input integer values. Pseudopolynomial algorithms are polynomial only when the integers are polynomially bounded in input size ($n^{O(1)}$). For knapsack with $W = 2^{60}$, $O(n \cdot W)$ is hopeless.

So 0-1 knapsack remains NP-hard – the DP doesn't actually break the hardness. The DP is useful when W is small (a few thousand, say), but fundamentally the difficulty sits in the *values* of the integers, not their count. Subset Sum has the same character (same DP, $O(n \cdot T)$, pseudopolynomial in target T). Chapter 3 returns to the strongly-polynomial-vs-pseudopolynomial-vs-exponential hierarchy when it formalizes Big-O.

8.3 Self-adjusting heuristics

A different flavor of heuristic: instead of approximating an answer, the algorithm *reorganizes its data structure over time* based on how it's being used, so common operations get faster.

Definition — Self-adjusting heuristic

A heuristic that modifies a data structure in response to observed access patterns. The structure starts in some default arrangement and evolves to match the workload. Splay trees, AVL rotations, LRU caches, move-to-front lists, and adaptive B-trees are all examples.

8.4 Worked example: move-to-front on a linear search

Example Move-to-front

Start with a list: `[17, 5, 88, 3, 42, 91, 76, 64]`.
Search for 42: linear scan takes 5 comparisons. After the search, *move 42 to the front.*
List is now: `[42, 17, 5, 88, 3, 91, 76, 64]`.
Search for 42 again: 1 comparison.
Search for 64: 8 comparisons, then move 64 to the front.

List: `[64, 42, 17, 5, 88, 3, 91, 76]`.
Search for 42: 2 comparisons.

Key idea Why it works

Real-world access patterns are rarely uniform. A small number of items are hot, most are cold. Move-to-front drifts the hot items toward the front of the list, where they're cheap to find. On a skewed workload – think the handful of popular queries in a search engine, or the same test scores being looked up hundreds of times – this can make an $O(n)$ linear search effectively $O(1)$ amortized.
On a *uniform* workload, move-to-front still works, it just doesn't help much. That's the right kind of tradeoff: it never makes access slower (per operation the list still takes $O(n)$ worst case) and it makes the common case much faster when the common case exists.

Gotcha Move-to-front is not a substitute for the right data structure

If the list is big and the workload isn't heavily skewed, a hash table (§later in course) is $O(1)$ average regardless of the access pattern and you should use one. Move-to-front is a nice $O(1)$-amortized tweak when a linked list is already the right structure for other reasons.

8.5 When to reach for a heuristic (and when not to)

Reach for a heuristic when	Use the exact algorithm when
The problem is NP-hard and inputs are large.	The problem has a polynomial-time exact algorithm (most of this course).
"Pretty good, fast" is more valuable than "optimal, slow."	Optimality is required by the spec (safety, financial, legal).
You can measure how close the heuristic gets to optimal and it's good enough.	You can't measure closeness and a bad answer has real consequences.
The algorithm will run on streaming / online data where you can't see the whole input.	The input is fully available and small enough to process exactly.

Key idea Big picture

Heuristics are not a failure of computer science – they are one of its honest responses to the real world. Some problems are too big to solve optimally; others arrive as a stream where you have to commit before seeing everything; many data structures get dramatically faster when allowed to reshape themselves around actual usage. The habit of mind to build here is: *ask what "good enough" means for this problem, pick a heuristic whose failure modes you understand, and measure how close to optimal it gets in practice.* That framing will serve you long after the specific greedy-knapsack or move-to-front trick stops mattering.

9 2.9 Greedy Algorithms

§2.8 introduced heuristics – the idea of accepting non-optimal answers for speed. A **greedy algorithm** is the most common style of heuristic: at each step, commit to the choice that looks best *right now*, without looking ahead. Sometimes this lands you at the globally optimal answer (change-making, fractional-knapsack, activity selection, shortest paths). Sometimes it doesn't (0-1 knapsack from §2.8). Knowing which problems

are in the first bucket and which aren't is the real skill.

Definition — Greedy algorithm

A **greedy algorithm** solves a problem by repeatedly making the *locally optimal* choice – the option that looks best at this step – and never reconsidering. It does not backtrack, cache alternatives, or look ahead. The algorithm terminates when no further choice is possible or the goal has been reached.

Definition — Locally optimal vs. globally optimal

A **locally optimal** choice is the best move at a single step considered in isolation. A **globally optimal** solution is the best overall outcome across all steps. Greedy algorithms make locally optimal choices and *hope* those add up to a global optimum. Whether they do depends on the problem.

9.1 Example 1: making change (and why US coins are special)

Example `MakeChange` with US denominations

```
MakeChange(amount):
    while amount >= 25: take a quarter; amount -= 25
    while amount >= 10: take a dime;    amount -= 10
    while amount >= 5:  take a nickel;  amount -= 5
    while amount >= 1:  take a penny;   amount -= 1
```

For 91 cents: 3 quarters (75), 1 dime (85), 1 nickel (90), 1 penny (91). Total: 6 coins. That also happens to be the fewest coins possible.

Gotcha Greedy change-making is NOT always optimal

It works for US coins – 1, 5, 10, 25 – because the denomination set has a property called *matroid-compatibility*; for every amount, grabbing the largest coin that fits is part of some optimal solution. Change the denominations and greedy breaks. Example: coins {1, 3, 4}, target 6. Greedy takes 4, then 1, then 1 – three coins. The optimal is 3 + 3 – two coins. Same algorithm, different coin set, different answer.

The exact-optimal solution for arbitrary denominations is dynamic programming, $O(n \cdot A)$ in the amount A.

9.2 Example 2: fractional knapsack – greedy is optimal

Definition — Fractional knapsack

The knapsack problem, but you may take any fraction $f \in [0, 1]$ of any item. Its weight contribution is $f \cdot w_i$ and its value contribution is $f \cdot v_i$.

Example Greedy by value-per-pound

```
FractionalKnapsack(items, capacity):
    Sort items descending by (value / weight)
    remaining = capacity
    for each item in sorted order:
        if item.weight <= remaining:
```

```
            take all of item
            remaining -= item.weight
        else:
            take fraction = remaining / item.weight of item
            remaining = 0
            break
```

This is **provably optimal**. Intuition: at every pound of knapsack space, you prefer the item with the highest dollars-per-pound. Allowing fractions means you can always *fill* the knapsack, so "best value density packed first" wins. 0-1 knapsack lacks this property because you're forced into an all-or-nothing choice at the last item.

Key idea The difference between 0-1 and fractional

Same items, same capacity, same greedy rule – but on 0-1 the greedy can leave optimal value on the table (see §2.8), while on fractional it can't. The lesson: a greedy rule's correctness depends on the problem structure, not just the rule. Always ask "is this problem one where greedy is known to be optimal?" before shipping code.

9.3 Example 3: activity selection – and why "earliest finish" wins

Definition — Activity selection problem

Given activities each with a start time and a finish time, pick the largest subset of activities such that no two chosen activities overlap. Equivalent to scheduling the maximum number of compatible meetings on a single room.

Example Greedy by earliest finish

```
ActivitySelection(A):
    Sort A ascending by finish time
    chosen = [A[0]]
    current_end = A[0].finish
    for i from 1 to n-1:
        if A[i].start >= current_end:
            chosen.append(A[i])
            current_end = A[i].finish
    return chosen
```

Aside Why earliest finish, not shortest or earliest start

Picking the shortest activity fails: a 10-minute activity that spans lunch blocks both morning and afternoon longer ones. Picking the earliest start fails for the same reason – a 9am-to-5pm activity beats out everything. **Earliest finish** leaves the maximum remaining room in the day for further choices, which is exactly what greedy needs to stay on track.

Example A vacation day worked by hand

Activities: museum (9–11), hike (10–12), boat (11–13), hang-glide (14–15), fireworks (19–20), nightclub (18–23).
Sorted by finish: museum, hike, boat, hang-glide, fireworks, nightclub.

- Choose museum (finishes 11).
- Hike starts 10 < 11, skip.
- Boat starts 11 $\geq$ 11, choose (finishes 13).
- Hang-glide starts 14 $\geq$ 13, choose (finishes 15).
- Fireworks starts 19 $\geq$ 15, choose (finishes 20).
- Nightclub starts 18 < 20, skip.

Chosen: museum, boat, hang-glide, fireworks. That's 4 activities, which is the maximum possible.

Example Why earliest-finish is optimal: a sketch

Activity selection is one of the few greedy algorithms with a clean, short correctness proof. The 2-step template from above made concrete:
Greedy-choice property. Some optimal solution includes the activity with the earliest finish time. *Why:* let $\mathcal{O}$ be any optimal schedule, and let a^* be the earliest-finish activity overall (call its finish time f^*). If $a^* \in \mathcal{O}$, done. Otherwise let $a' \in \mathcal{O}$ be the activity in $\mathcal{O}$ with the earliest finish time. Since a^* has the globally earliest finish, $f^* \leq f(a')$. Swap a' out and a^* in: the rest of $\mathcal{O}$ all started *after* $f(a') \geq f^*$, so they're still compatible with a^*. The new schedule has the same number of activities, includes a^*, and is also optimal. This is an **exchange argument** – the standard shape of a greedy-choice proof.
Optimal substructure. After picking a^*, the remaining problem is: pick the most activities from those that start at or after f^*. That subproblem has the same shape as the original (still an activity-selection problem, on a smaller set). Combining the greedy choice a^* with an optimal solution to the remaining subproblem gives an optimal solution to the whole.
Putting them together. Greedy-choice says picking a^* loses nothing. Optimal substructure says we can recurse on the remaining problem with the same algorithm. By induction on input size, the algorithm produces an optimal schedule.
This is the entire shape of every greedy-correctness proof you will write: an exchange argument for greedy-choice, plus a same-shape argument for optimal substructure. Compare to the longer DP proofs in §2.10 – greedy is simpler when it applies, and that simplicity is exactly why we look for greedy-friendly structure first.

9.4 When greedy actually works

There is a pattern behind these examples. Greedy is provably optimal on problems with one of two underlying structures:

- **Matroid structure.** Fancy name, simple idea: if the locally optimal choice is always part of *some* globally optimal solution, and "independent subsets" have an exchange property. Change-making with US coins, minimum-spanning-tree (Kruskal, Prim, later in course), and various scheduling problems have it.
- **Greedy-choice property + optimal substructure.** Same two ingredients that justify dynamic programming, minus the need to enumerate every subproblem. Activity selection fits here.

Key idea Two moves of a correctness argument

To argue a greedy algorithm is optimal you usually show:

1. **Greedy-choice property.** There exists an optimal solution that begins with the greedy choice. ("Swapping in the greedy item doesn't hurt.")
2. **Optimal substructure.** After making the greedy choice, the remaining problem has the same shape, and its optimal solution can be extended to a global optimum.

Neither condition is obvious to verify; proving greedy correctness is genuinely harder than writing the algorithm.

9.5 Greedy vs. other strategies

Strategy	How it decides	Cost / guarantee
Greedy	Best local choice, no reconsideration	Fast; optimal only on certain problems
Dynamic programming	Remember optimal answers to subproblems, build up	$O(n \cdot \text{state size})$; optimal when subproblems overlap
Divide and conquer	Split, recurse, combine	Optimal; works when subproblems are independent
Backtracking	Try choices, undo bad ones	Exponential worst case but often prunable
Branch & bound	Backtracking + bound pruning	Exact; can be practical for small NP-hard inputs

Gotcha The trap: assuming greedy works

The most common mistake is to *assume* the greedy algorithm you wrote is optimal because it produced the right answer on the example you tested. Try adversarial inputs. Read whether the textbook problem is known to admit a greedy-optimal solution. When in doubt, dynamic programming is usually within reach and provably optimal.

Key idea Big picture

Greedy is the first algorithm-design strategy worth internalizing because it is often the right answer and always worth trying first. Write down the most natural "best-at-each-step" rule, test it against an optimal solution on a few adversarial inputs, and either (a) ship it with a proof sketch of why it's optimal, or (b) observe it fail and fall back to DP or exact search. The three problems in this section – change, fractional knapsack, activity selection – are worth keeping in your head as the canonical "greedy wins" references, and 0-1 knapsack from §2.8 as the canonical "greedy loses" reference.

10 2.10 Dynamic Programming

Greedy (§2.9) makes one choice at each step and never looks back. Pure recursion (§2.5) recomputes the same subproblems over and over. **Dynamic programming (DP)** splits the difference: it solves every subproblem *once*, stores the answer, and assembles the final answer from those stored pieces.

When the problem has overlapping subproblems and optimal substructure, DP turns exponential-time recurrences into polynomial algorithms – usually a dramatic improvement.

Definition — Dynamic programming

A problem-solving technique that splits a problem into smaller subproblems, **computes each subproblem's answer exactly once**, stores it in a table, and uses the stored answers to build up the final answer. Two common forms: **memoization** (top-down recursion with a cache) and **tabulation** (bottom-up iteration filling a table).

Definition — Overlapping subproblems

A recurrence has **overlapping subproblems** when the same subproblem is computed multiple times in the naive recursion tree. DP's payoff is skipping those repeated computations.

Definition — Optimal substructure

A problem has **optimal substructure** when an optimal solution to the whole can be assembled from optimal solutions to its subproblems. Without this, storing subproblem answers doesn't help.

10.1 Example 1: Fibonacci, fixed

§2.5 showed naive recursive `fib` blowing up exponentially because `fib(n-1)` and `fib(n-2)` recompute overlapping work. DP fixes this two ways.

Top-down memoization.. Keep a cache, check it before recursing:

Example Memoized recursion

```
#include <vector>

std::vector<long long> memo;  // sized to n+1, filled with -1

long long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
```

Each `fib(k)` gets computed once. $O(n)$ time, $O(n)$ memory on the cache plus $O(n)$ on the call stack.

Bottom-up tabulation.. Build the answers iteratively from the base cases up:

Example Tabulated iteration

```
long long fib(int n) {
    if (n <= 1) return n;
    long long prev = 0, cur = 1;
    for (int i = 2; i <= n; ++i) {
        long long next = prev + cur;
        prev = cur;
        cur = next;
    }
```

```
    return cur;
}
```

$O(n)$ time, $O(1)$ memory, no recursion. This is usually the preferred form when the dependency structure is simple.

Key idea Top-down vs. bottom-up

Top-down memoization is easier to write when a recurrence is already in your head – you write the natural recursion, add three lines of caching, and stop. Bottom-up is usually tighter, avoids recursion depth limits, and often lets you throw away old table entries you no longer need (**rolling-array** trick – the Fibonacci version uses only two variables instead of an n-size array).

10.2 Example 2: longest common substring (LCS)

Aside "LCS" usually means subsequence, not substring

Heads up on naming. The acronym **LCS** in most textbooks (CLRS Ch. 15.4, MIT 6.006 Lecture 16, every DP reference you'll Google) stands for **longest common *subsequence*** – a non-contiguous match where characters from s and t must appear in the same order but can have gaps in between. So `ACE` is a common subsequence of `ABCDE` and `XAYCEZ`.

What *this* section covers is the longest common **substring** (contiguous) variant – a different problem with a different recurrence:

	Substring (this section)	**Subsequence (standard "LCS")**
Match must be	contiguous	in-order, gaps allowed
$L[i][j]$ if mismatch	0 (reset)	$\max(L[i-1][j],\ L[i][j-1])$
Answer location	max anywhere in table	$L[m][n]$ corner

When you see "LCS" outside this chapter – in a textbook, interview question, or Stack Overflow answer – assume *subsequence* unless context says otherwise. Both variants are useful; just don't confuse them.

A classic DP where the benefit is overwhelming. Naive recursion is exponential; DP is $O(n \cdot m)$ in the two string lengths. This algorithm underlies real-world tools: DNA sequence alignment, git's rename detection, plagiarism detection.

Definition — Longest common substring problem

Given two strings `s` and `t`, find the longest *contiguous* substring that appears in both. (Distinct from longest common *subsequence*, where characters don't have to be adjacent.)

The recurrence.. Let $L[i][j]$ be the length of the longest common substring that *ends at* `s[i]` and `t[j]`.

$$L[i][j] = \begin{cases} 0 & \text{if } s[i] \neq t[j] \\ 1 + L[i-1][j-1] & \text{if } s[i] = t[j] \end{cases}$$

The answer is the maximum value anywhere in the table. Its position tells you where the substring ends in `s`, which together with the length gives you the substring itself.

Example LCS algorithm

```
#include <string>
#include <vector>

std::string lcs(const std::string& s, const std::string& t) {
    int n = s.size(), m = t.size();
    std::vector<std::vector<int>> L(n, std::vector<int>(m, 0));

    int best_len = 0, best_row = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (s[i] == t[j]) {
                L[i][j] = (i > 0 && j > 0) ? L[i-1][j-1] + 1 : 1;
                if (L[i][j] > best_len) {
                    best_len = L[i][j];
                    best_row = i;
                }
            }
        }
    }
    int start = best_row - best_len + 1;
    return s.substr(start, best_len);
}
```

$O(n \cdot m)$ time, $O(n \cdot m)$ memory. For typical inputs (dozens to thousands of characters) this is fine.

Worked trace: "Look" vs "Books".. Rows indexed by Look, columns by Books.

	B	o	o	k	s
L	0	0	0	0	0
o	0	1	1	0	0
o	0	1	2	0	0
k	0	0	0	3	0

The maximum is 3, in row 3 (the k) column 3. The substring ends at `s[3] = 'k'` and is 3 long: "ook".

10.3 Space optimization: the rolling-array trick

The LCS recurrence only looks one row back. So the whole table is wasteful – you only need the previous row and the current row.

Example Two-row LCS

```
std::vector<int> prev(m, 0), cur(m, 0);
int best_len = 0, best_row = 0;
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        if (s[i] == t[j])
            cur[j] = (j > 0 ? prev[j-1] : 0) + 1;
        else
```

```
            cur[j] = 0;
        if (cur[j] > best_len) { best_len = cur[j]; best_row = i; }
    }
    std::swap(prev, cur);
    std::fill(cur.begin(), cur.end(), 0);
}
```

Same $O(n \cdot m)$ time. Memory drops from $O(n \cdot m)$ to $O(m)$. On the billion-character DNA strings the textbook mentions, this distinction is "runs in memory" vs. "doesn't fit in RAM."

10.4 Recognizing DP problems

Not every recursion wants memoization. A few signals that DP is the right tool:

- The naive recurrence recomputes the same subproblem on multiple branches (Fibonacci, edit distance, LCS, coin change, knapsack).
- The problem has a clear *state* – a small tuple of integers (index into a string, remaining capacity, whether you're allowed to take an item, etc.) – and the answer depends only on that state.
- An optimal answer to the whole can be built by combining optimal answers to smaller states.

If the recursion tree doesn't overlap (binary search, merge sort), plain recursion is fine and DP adds overhead for no benefit. If the problem lacks optimal substructure (many game-theoretic or greedy-fails problems), DP doesn't apply and you need a different technique.

Gotcha DP is not automatic – designing the state is the hard part

"Apply dynamic programming" sounds like a step, but the real work is identifying *what subproblem to solve*. Most DP bugs come from a state definition that either misses necessary information (answer depends on something you didn't track) or includes unnecessary information (table blows up exponentially). Spend the thinking up front defining $L[i][j] =$ *"the _____"* precisely; the code is almost mechanical after that.

10.5 DP vs. greedy vs. divide-and-conquer

	Greedy	DP
Decision style	One choice per step, no backtrack	Considers all choices for each subproblem
Subproblems overlap	(doesn't matter)	Required for DP to help
Optimal substructure needed	yes	yes
Guarantees	Optimal only for some problems	Always optimal when it applies
Typical cost	$O(n)$ or $O(n \log n)$	Product of state dimensions

Aside Divide and conquer is the third cousin

Divide-and-conquer (merge sort, binary search, Karatsuba) also breaks a problem into subproblems, but the subproblems are *independent* – no need to cache, no overlap. When the recurrence happens to produce *disjoint* subproblems, D&C is the natural form; when it produces *overlapping* ones, DP earns its keep.

Key idea Big picture

DP is the natural upgrade from recursion when the recurrence tree starts exploding. Every DP solution answers one question: "what state do I need to track so that the optimal answer depends only on that state and the answers to smaller states?" Once you've written that down, the rest is filling a table. The two examples in this section – Fibonacci (one-dimensional state) and LCS (two-dimensional state) – are worth keeping in your head as the shape of every DP solution you'll meet later. Coin change, knapsack, edit distance, and most string-alignment problems follow the same template.

What this connects to

Aside Forward links to later chapters

- **Chapter 3 (Big-O).** The informal "naïve Fibonacci is exponential, memoized is linear" claim becomes the precise statement $O(2^n)$ vs. $O(n)$. Same story for every algorithmic claim you made in this chapter – ch. 3 supplies the notation.
- **Sorting in ch. 3.** Insertion and selection sort are incremental; merge sort and quicksort are divide-and-conquer (recursive); the runtime analyses reuse the recursion-tree reasoning from 2.5 verbatim.
- **Binary search** (2.5) will reappear as the inner loop of BST search (ch. 6) and as the `std::lower_bound` primitive used everywhere.
- **Dynamic programming** (2.10) resurfaces in several later-course problems: longest-path-in-DAG, knapsack, edit distance, string alignment – all use the same "define the state, write the recurrence, fill the table" workflow.
- **Greedy** (2.9) is the engine behind Dijkstra's algorithm and Kruskal's MST (optional ch. 10), and behind Huffman coding in compression. The greedy-choice property test you practiced here is exactly what justifies those algorithms' correctness.
- **Ethics / privacy** (2.6-2.7) isn't a detour – it's the lens through which the course will ask you to evaluate every project. Expect discussion-board prompts on this all semester.

Companion materials. One-page cheat sheet: `chapters/ch_2/notes.tex`. Practice prompts (problem → pseudocode → C++ workflow): `chapters/ch_2/practice.md`.